🏠 House Price Prediction using Python

Exploratory Data Analysis (EDA) and Machine Learning

📌 Project Overview

This project explores the Boston Housing dataset to understand the factors influencing house prices. Exploratory Data Analysis (EDA) is performed to uncover trends, relationships, and patterns within the data. A Linear Regression model is then built to predict house prices based on various housing characteristics.

The project demonstrates the complete data analysis workflow, including data loading, cleaning, visualization, model building, and performance evaluation using Python.

🎯 Project Objectives

- Load and explore the Boston Housing dataset.
- Understand the dataset structure and variables.
- Perform Exploratory Data Analysis (EDA).
- Visualize relationships between important variables.
- Build a Linear Regression model for house price prediction.
- Evaluate the model using regression performance metrics.

📚 Import Required Libraries

📂 Load the Dataset

The dataset is loaded into a Pandas DataFrame. It contains information about various housing characteristics such as crime rate, number of rooms, tax rate, accessibility to highways, and the median value of owner-occupied homes (MEDV).

👀 Display the First Five Rows

The first five rows provide an overview of the dataset and help verify that it has been loaded correctly.

ℹ️ Dataset Information

The `.info()` function provides information about the dataset, including the number of observations, data types, and whether there are any missing values.

📊 Descriptive Statistics

Summary statistics help us understand the distribution of numerical features by displaying values such as the mean, minimum, maximum, and quartiles.

🔍 Missing Value Analysis

Checking for missing values ensures the dataset is complete before performing further analysis or building machine learning models.

📐 Dataset Shape

The shape of the dataset shows the total number of rows and columns available for analysis.

🔥 Correlation Heatmap

A correlation heatmap is used to identify relationships between numerical variables. Positive correlations are shown in warmer colors, while negative correlations appear in cooler colors. This helps identify features that have strong relationships with house prices.

📈 Distribution of House Prices

The histogram below illustrates how house prices are distributed throughout the dataset. This visualization helps identify skewness, peaks, and possible outliers.

📉 Relationship Between Average Rooms and House Price

The scatter plot below shows the relationship between the average number of rooms (RM) and the median house value (MEDV). This visualization helps determine whether larger houses tend to have higher prices.

🎯 Feature Selection

The target variable for this project is **MEDV**, which represents the median value of owner-occupied homes.

The remaining variables are used as input features for the prediction model.

🎌 Splitting the Dataset

The dataset is divided into training and testing sets. The training set is used to train the model, while the testing set is used to evaluate its predictive performance on unseen data.

🤖 Model Training

A Linear Regression model is trained using the training dataset to learn the relationship between housing features and house prices.

📊 Model Evaluation

The trained model is evaluated using the following performance metrics:

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)
- R^2 Score

These metrics measure how accurately the model predicts house prices.

📉 Actual vs Predicted House Prices

The scatter plot below compares the actual house prices with the predicted values generated by the Linear Regression model. A good model should produce predictions that closely align with the actual values.

In [3]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)
```

```
# Load dataset
df = pd.read_csv("4) house Prediction Data Set.csv")

# Display first five rows
df.head()
```

Out[3]:

	0.00632	18.00	2.310	0	0.5380	6.5750	65.20	4.0900	1	296.0	15.30	396.90	4.98	24.00
0								0.02731	0.00	7.070	0	0.4690	6.4210	78...
1								0.02729	0.00	7.070	0	0.4690	7.1850	61...
2								0.03237	0.00	2.180	0	0.4580	6.9980	45...
3								0.06905	0.00	2.180	0	0.4580	7.1470	54...
4								0.02985	0.00	2.180	0	0.4580	6.4300	58...

In [4]:
```
df.head()
```

Out[4]:

	0.00632	18.00	2.310	0	0.5380	6.5750	65.20	4.0900	1	296.0	15.30	396.90	4.98	24.00
0								0.02731	0.00	7.070	0	0.4690	6.4210	78...
1								0.02729	0.00	7.070	0	0.4690	7.1850	61...
2								0.03237	0.00	2.180	0	0.4580	6.9980	45...
3								0.06905	0.00	2.180	0	0.4580	7.1470	54...
4								0.02985	0.00	2.180	0	0.4580	6.4300	58...

In [5]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 505 entries, 0 to 504
Data columns (total 1 columns):
 #   Column                                                                                                        Non-Null Count
Dtype
---  ------                                                                                                        --------------
-----
 0    0.00632  18.00   2.310  0  0.5380  6.5750  65.20  4.0900   1  296.0  15.30 396.90   4.98  24.00  505 non-null
str
dtypes: str(1)
memory usage: 4.1 KB
```

In [6]:
```
df.describe()
```

Out[6]:

	0.00632 18.00 2.310 0 0.5380 6.5750 65.20 4.0900 1 296.0 15.30 396.90 4.98 24.00
count	505
unique	505
top	0.02731 0.00 7.070 0 0.4690 6.4210 78...
freq	1

In [7]:
```
df.isnull().sum()
```

Out[7]:
```
0.00632  18.00   2.310  0  0.5380  6.5750  65.20  4.0900   1  296.0  15.30 396.90   4.98  24.00    0
dtype: int64
```

In [8]:
```
df.shape
```

Out[8]:
```
(505, 1)
```

In [9]:
```
df = pd.read_csv("4) house Prediction Data Set.csv", sep=",")
df.head()
```

Out[9]:

	0.00632	18.00	2.310	0	0.5380	6.5750	65.20	4.0900	1	296.0	15.30	396.90	4.98	24.00
0								0.02731	0.00	7.070	0	0.4690	6.4210	78...
1								0.02729	0.00	7.070	0	0.4690	7.1850	61...
2								0.03237	0.00	2.180	0	0.4580	6.9980	45...
3								0.06905	0.00	2.180	0	0.4580	7.1470	54...
4								0.02985	0.00	2.180	0	0.4580	6.4300	58...

In [10]:
```
df = pd.read_csv("4) house Prediction Data Set.csv", sep=";")
df.head()
```

Out[10]:

	0.00632	18.00	2.310	0	0.5380	6.5750	65.20	4.0900	1	296.0	15.30	396.90	4.98	24.00
0								0.02731	0.00	7.070	0	0.4690	6.4210	78...
1								0.02729	0.00	7.070	0	0.4690	7.1850	61...
2								0.03237	0.00	2.180	0	0.4580	6.9980	45...
3								0.06905	0.00	2.180	0	0.4580	7.1470	54...
4								0.02985	0.00	2.180	0	0.4580	6.4300	58...

In [14]:
```
df.shape
```

Out[14]: (506, 14)

In [15]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 506 entries, 0 to 505
Data columns (total 14 columns):
 #   Column  Non-Null Count  Dtype  
---  ------  --------------  -----  
 0   0       506 non-null    float64
 1   1       506 non-null    float64
 2   2       506 non-null    float64
 3   3       506 non-null    int64  
 4   4       506 non-null    float64
 5   5       506 non-null    float64
 6   6       506 non-null    float64
 7   7       506 non-null    float64
 8   8       506 non-null    int64  
 9   9       506 non-null    float64
 10  10      506 non-null    float64
 11  11      506 non-null    float64
 12  12      506 non-null    float64
 13  13      506 non-null    float64
dtypes: float64(12), int64(2)
memory usage: 55.5 KB
```

🏷️ Assign Column Names

The dataset was loaded without predefined column headers. Appropriate feature names are assigned based on the Boston Housing dataset documentation to improve readability and facilitate analysis.

In [16]:
```
df.columns = [
    "CRIM", "ZN", "INDUS", "CHAS", "NOX", "RM",
    "AGE", "DIS", "RAD", "TAX", "PTRATIO",
    "B", "LSTAT", "MEDV"
]

df.head()
```

Out[16]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	MEDV
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296.0	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242.0	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242.0	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222.0	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222.0	18.7	396.90	5.33	36.2

In [17]:
```
df.describe()
```

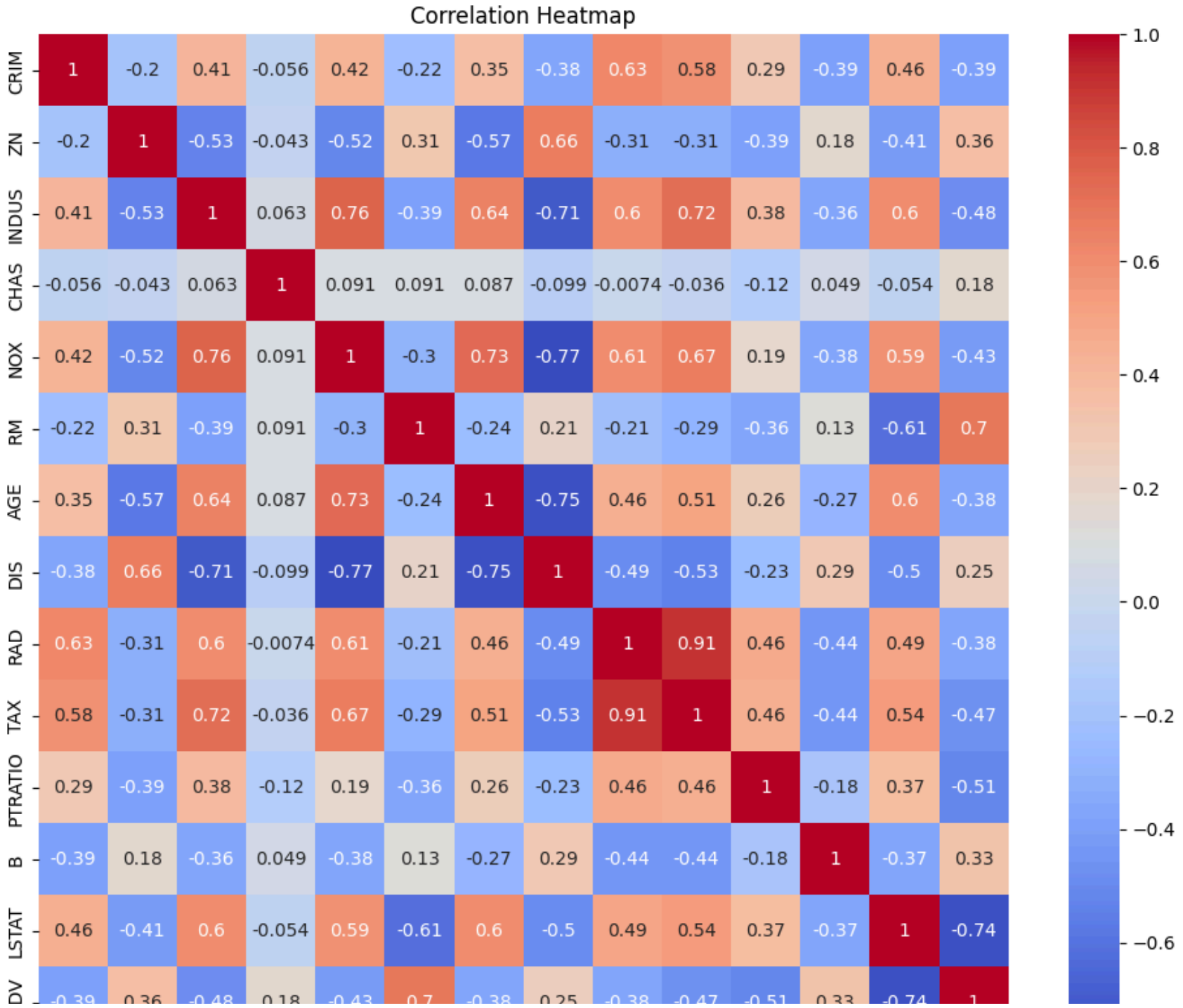
Out[17]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX
count	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000
mean	3.613524	11.363636	11.136779	0.069170	0.554695	6.284634	68.574901	3.795043	9.549407	408.237154
std	8.601545	23.322453	6.860353	0.253994	0.115878	0.702617	28.148861	2.105710	8.707259	168.537116
min	0.006320	0.000000	0.460000	0.000000	0.385000	3.561000	2.900000	1.129600	1.000000	187.000000
25%	0.082045	0.000000	5.190000	0.000000	0.449000	5.885500	45.025000	2.100175	4.000000	279.000000
50%	0.256510	0.000000	9.690000	0.000000	0.538000	6.208500	77.500000	3.207450	5.000000	330.000000
75%	3.677083	12.500000	18.100000	0.000000	0.624000	6.623500	94.075000	5.188425	24.000000	666.000000
max	88.976200	100.000000	27.740000	1.000000	0.871000	8.780000	100.000000	12.126500	24.000000	711.000000

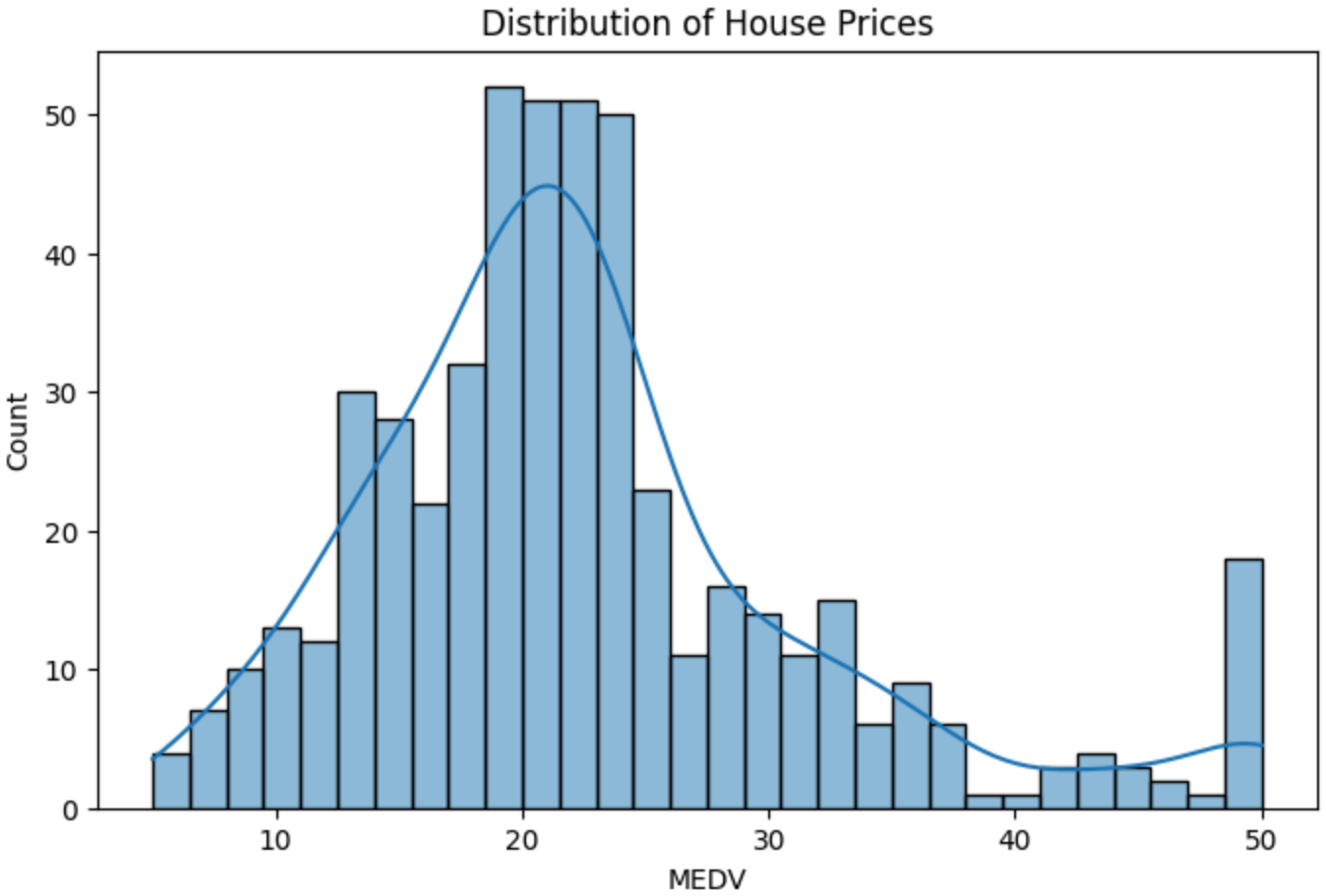
In [18]:
```
df.isnull().sum()
```

Out[18]:
```
CRIM       0
ZN         0
INDUS      0
CHAS       0
NOX        0
RM         0
AGE        0
DIS        0
RAD        0
TAX        0
PTRATIO    0
B          0
LSTAT      0
MEDV       0
dtype: int64
```

In [19]:
```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12,10))
sns.heatmap(df.corr(), annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```

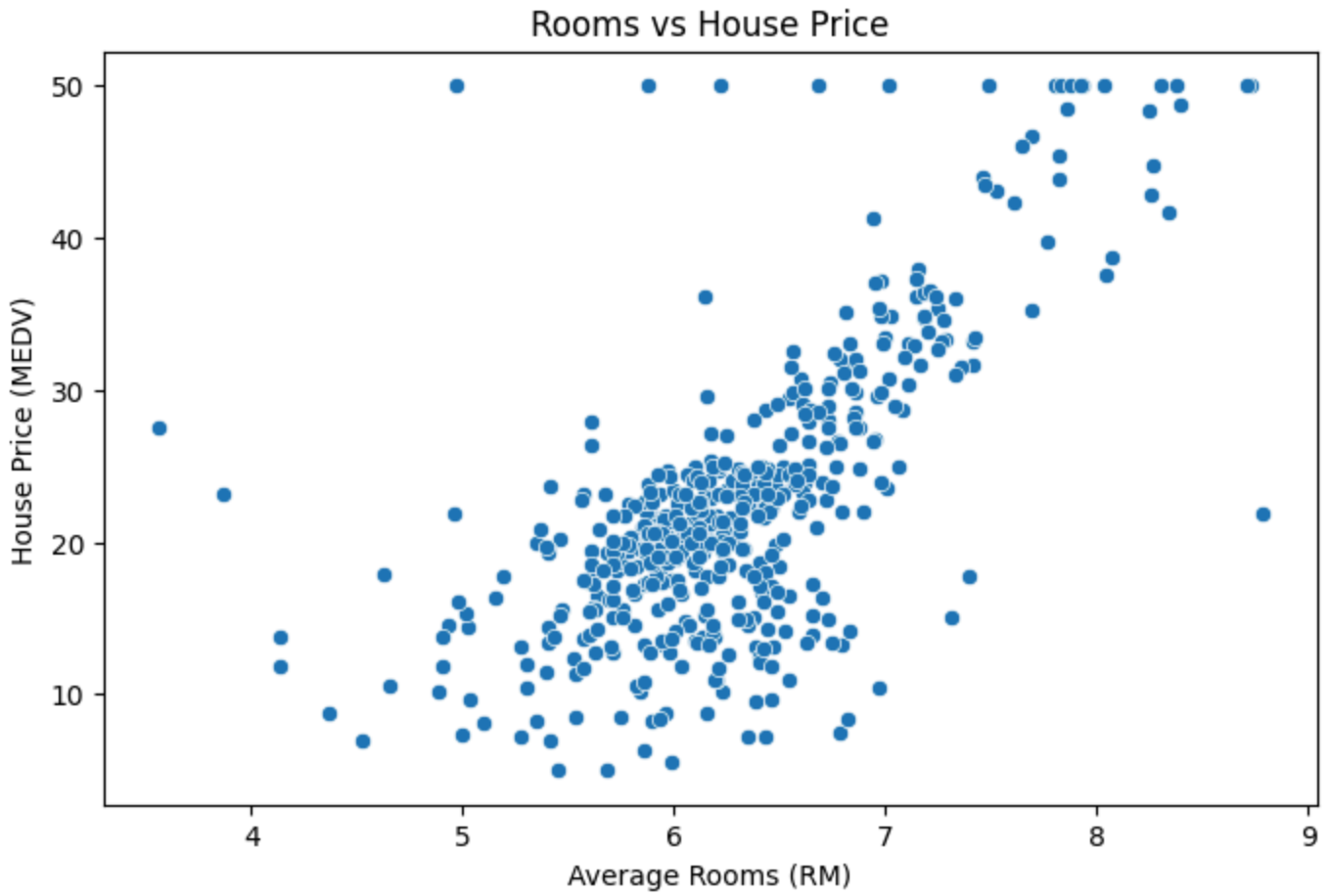

Correlation Heatmap
CRIM 1 -0.2 0.41 -0.056 0.42 -0.22 0.35 -0.38 0.63 0.58 0.29 -0.39 0.46 -0.39
ZN -0.2 1 -0.53 -0.043 -0.52 0.31 -0.57 0.66 -0.31 -0.31 -0.39 0.18 -0.41 0.36
INDUS 0.41 -0.53 1 0.063 0.76 -0.39 0.64 -0.71 0.6 0.72 0.38 -0.36 0.6 -0.48
CHAS -0.056 -0.043 0.063 1 0.091 0.091 0.087 -0.099 -0.0074 -0.036 -0.12 0.049 -0.054 0.18
NOX 0.42 -0.52 0.76 0.091 1 -0.3 0.73 -0.77 0.61 0.67 0.19 -0.38 0.59 -0.43
RM -0.22 0.31 -0.39 0.091 -0.3 1 -0.24 0.21 -0.21 -0.29 -0.36 0.13 -0.61 0.7
AGE 0.35 -0.57 0.64 0.087 0.73 -0.24 1 -0.75 0.46 0.51 0.26 -0.27 0.6 -0.38
DIS -0.38 0.66 -0.71 -0.099 -0.77 0.21 -0.75 1 -0.49 -0.53 -0.23 0.29 -0.5 0.25
RAD 0.63 -0.31 0.6 -0.0074 0.61 -0.21 0.46 -0.49 1 0.91 0.46 -0.44 0.49 -0.38
TAX 0.58 -0.31 0.72 -0.036 0.67 -0.29 0.51 -0.53 0.91 1 0.46 -0.44 0.54 -0.47
PTRATIO 0.29 -0.39 0.38 -0.12 0.19 -0.36 0.26 -0.23 0.46 0.46 1 -0.18 0.37 -0.51
B -0.39 0.18 -0.36 0.049 -0.38 0.13 -0.27 0.29 -0.44 -0.44 -0.18 1 -0.37 0.33
LSTAT 0.46 -0.41 0.6 -0.054 0.59 -0.61 0.6 -0.5 0.49 0.54 0.37 -0.37 1 -0.74
DV -0.39 0.36 -0.48 0.18 -0.43 0.7 -0.38 0.25 -0.38 -0.47 -0.51 0.33 -0.74 1
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4
−0.6

In [20]:
```
plt.figure(figsize=(8,5))
sns.histplot(df["MEDV"], bins=30, kde=True)
plt.title("Distribution of House Prices")
plt.xlabel("MEDV")
plt.show()
```



In [21]:
```
plt.figure(figsize=(8,5))
sns.scatterplot(x=df["RM"], y=df["MEDV"])
plt.title("Rooms vs House Price")
```

```
plt.xlabel("Average Rooms (RM)")
plt.ylabel("House Price (MEDV)")
plt.show()
```



In [23]:
```
from sklearn.model_selection import train_test_split

X = df.drop("MEDV", axis=1)
y = df["MEDV"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
```

```
    random_state=42
)
```

In [24]:
```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)
```

Out[24]:

LinearRegression

▶ Parameters

▶ Fitted attributes

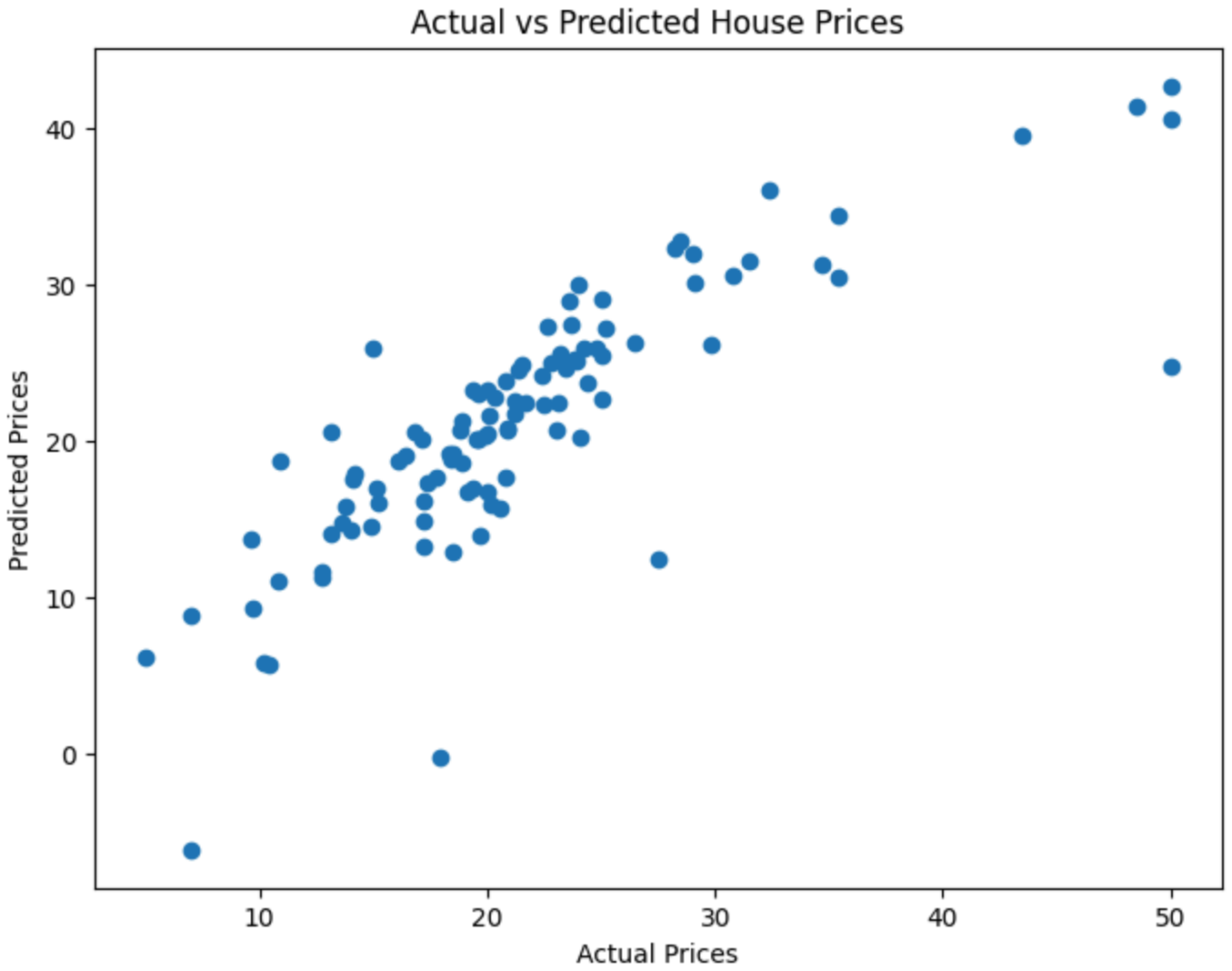
In [25]:
```
y_pred = model.predict(X_test)

y_pred[:10]
```

Out[25]:
```
array([28.99672362, 36.02556534, 14.81694405, 25.03197915, 18.76987992,
       23.25442929, 17.66253818, 14.34119   , 23.01320703, 20.63245597])
```

In [26]:
```
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = mse ** 0.5
r2 = r2_score(y_test, y_pred)

print("MAE :", mae)
print("MSE :", mse)
print("RMSE:", rmse)
print("R2 Score:", r2)
```

```
MAE : 3.1890919658878416
MSE : 24.291119474973538
RMSE: 4.928602182665339
R2 Score: 0.6687594935356317
```

In [27]:
```
import matplotlib.pyplot as plt

plt.figure(figsize=(8,6))
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted House Prices")
plt.show()
```



📈 Model Performance Summary

The Linear Regression model demonstrated good predictive performance on the testing dataset.

The evaluation metrics indicate that the model captures a significant proportion of the variance in house prices. Although Linear Regression serves as a strong baseline model, more advanced algorithms such as Random Forest or Gradient Boosting could

further improve prediction accuracy.

In [28]:
```
import joblib

joblib.dump(model, "house_price_model.pkl")
```

Out[28]:
```
['house_price_model.pkl']
```

✅ Conclusion

This project successfully explored the Boston Housing dataset and developed a Linear Regression model for predicting house prices.

Key Findings

- The dataset contained no missing values.
- Exploratory Data Analysis revealed important relationships among housing features.
- The number of rooms showed a strong positive relationship with house prices.
- Correlation analysis helped identify the most influential variables.
- The Linear Regression model achieved reliable predictive performance using the selected features.

Overall, this project demonstrates a complete end-to-end machine learning workflow, from data exploration and visualization to model development and evaluation.

🛠️ Technologies Used

- Python
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-learn

- Jupyter Notebook

🚀 Future Improvements

- Experiment with advanced regression models such as Random Forest and XGBoost.
- Perform feature engineering to improve prediction accuracy.
- Apply hyperparameter tuning and cross-validation.
- Deploy the trained model as an interactive web application using Streamlit or Flask.

👩‍💻 Author

Bolaji Odulate

Data Analyst | Python | SQL | Excel | Power BI

This project was developed as part of my Data Analytics Portfolio to demonstrate skills in Exploratory Data Analysis, Data Visualization, and Machine Learning using Python.

In [ ]: